Stock Analysis Dashboard · Technical Documentation

Project: StockAnalysisProject / WGU
Date: March 27, 2026
Version: 0.1.0

Table of Contents

1 System Overview
2 Architecture
3 Technology Stack
4 Backend (FastAPI)
5 Frontend (React)
6 Data Pipeline
7 Context Modules
8 Agent System (Ollama)
9 External APIs and Services
10 Database Schemas
11 Design System
12 API Reference
13 Configuration and Environment
14 Changelog · Recent Session Work

---

1. System Overview

The Stock Analysis Dashboard is a full-stack financial analytics platform that provides:

- **Multi-market universe**: Coverage of US (NASDAQ 100, S&P 500), Indian (NSE, BSE), UK (LSE FTSE 100), global indices, forex pairs, commodities, and Indian mutual funds
- **Real-time and historical charting**: Bloomberg-style terminal charts with OHLC candles, mountain/line/hybrid views, technical indicators (EMA20, SMA50, SMA200), and pitchfork analysis
- **Portfolio management**: Multi-portfolio tracking with transaction ledger, fee estimation, tax summaries, and AI copilot
- **Watchlists**: Custom watchlists with batch quoting, cron-based automation, and insights
- **Research lab**: ML signal generation, consumer correlation analysis, and contextual research
- **AI integration**: Ollama-powered analysis, download agents, and context agents
- **Wikipedia enrichment**: Automated Wikipedia title/extract retrieval for instrument descriptions
- **Scheduler**: APScheduler-based background jobs for EOD updates, intraday quotes, and batch operations

---

2. Architecture

```
+-------------------+       +-------------------+
+---------------------+
|   React Frontend  | <---> |  FastAPI Backend   | <---> |
External Services  |
|   (Port 3000)     |  REST |  (Port 8000)       |       |
           |
```

```
+-------------------+  /WS  +-------------------+
+---------------------+
        |                            |                          |
        |                     +------+------+             Yahoo
Finance
   ApexCharts               |             |             Wikipedia
API
   Recharts            SQLite DBs    Parquet Files      NSE/BSE
APIs
                       (ohlc.sqlite)  (local_market_    AMFI /
MFapi
                       (rag.sqlite)    data/)           GoodReturns
                       (scheduler)                      Ollama
(local)
                                                        Reddit /
Finnhub
```

Data Flow

1. **Universe Build**: `fetch_all_tickers.py` scrapes NSE/BSE/Wikipedia/PyTickerSymbols to produce `all_global_tickers.json`
2. **Download Pipeline**: `market_download.py` fetches OHLC from Yahoo Finance into Parquet files + SQLite
3. **API Layer**: `backend/app.py` reads local data (Parquet/SQLite), falls back to live Yahoo Finance
4. **Frontend**: React app calls REST endpoints, renders charts via ApexCharts, manages state via `useQuantEngine` hook
5. **Enrichment**: Wikipedia client enriches instrument metadata; consumer/news/open context modules provide additional layers
6. **Scheduler**: APScheduler runs background jobs for periodic data refresh

---

3. Technology Stack

Backend (Python 3.9+)

| Package | Purpose | |---------|---------| | FastAPI | REST API framework | | Uvicorn | ASGI server | | Pydantic | Request/response validation | | yfinance | Yahoo Finance data fetcher | | pandas / numpy | Data manipulation | | pyarrow | Parquet file I/O | | SQLAlchemy | Database ORM (scheduler job store) | | APScheduler | Background task scheduling | | feedparser | RSS feed parsing | | beautifulsoup4 / lxml | HTML/XML parsing | | pytickersymbols | Global index constituents | | exchange_calendars / holidays | Market session awareness | | requests | HTTP client | | tqdm | Progress bars |

Frontend (Node.js / React 19)

| Package | Purpose | |---------|---------| | react / react-dom 19 | UI framework | | react-scripts 5 | Build tooling (CRA) | | apexcharts / react-apexcharts | Interactive charting | | recharts | Supplementary charts | | web-vitals | Performance monitoring |

Infrastructure

- **Database**: SQLite (ohlc.sqlite, rag.sqlite, scheduler_jobs.sqlite)

- **File storage**: Parquet files under `local_market_data/{interval}/`
- **AI runtime**: Ollama (localhost:11434) for chat/embeddings
- **No cloud dependencies**: Entirely self-hosted

---

4. Backend (FastAPI)

File: `backend/app.py`

The main application file (~2200 lines) contains:

- **CORS configuration**: Allows all origins for local development
- **Startup/shutdown hooks**: Logger initialization, APScheduler startup
- **SQLite helpers**: `_ensure_ohlc_sqlite()`, `_info_db_path()`, schema creation
- **OHLC data reading**: `_read_local_series()` tries Parquet first, falls back to SQLite, then live Yahoo
- **Parquet compatibility**: Uses `parquet_symbol_key()` for filesystem-safe filenames, with fallback to raw symbol paths
- **Instrument info caching**: `_fetch_and_cache_instrument_info()` merges Yahoo info with Wikipedia enrichment
- **Wikipedia integration**: `_wiki_for_ticker()` checks cache first, falls back to live Wikipedia API
- **Numeric safety**: `_safe_round()` handles NaN, None, and non-numeric values throughout

File: `backend/scheduler.py`

APScheduler with SQLAlchemy job store:

- **EOD update job**: Downloads daily bars for all universe symbols
- **Intraday session job**: 15-minute bars during market hours
- **Live quote cache**: Batch fetches current prices for portfolio/watchlist symbols
- **Exchange-aware scheduling**: Uses `exchange_calendars` to skip non-trading days

Key Backend Functions

- `get_ticker_details(symbol)` · Returns comprehensive instrument metadata including Wikipedia enrichment, safe numeric handling, and news
- `get_ohlc_data(symbol, timeframe)` · Returns OHLC series with local-first resolution
- `download_ticker_data_on_demand(symbol)` · On-demand Parquet download
- `_run_bulk_info_job(job_id, body)` · Background bulk instrument info loading
- `search_instruments(query)` · Full-text search across universe + Yahoo Finance API

---

5. Frontend (React)

Application Structure

```
src/
  App.js                    # Root component, module routing,
sidebar
  index.js                  # Entry point
```

```
  hooks/
    useQuantEngine.js       # Central state management hook (~2500
lines)
  pages/
    HomeDashboard/          # Landing page with market overview
    PortfolioDashboard/     # Portfolio management with AI copilot
    UniversePage/           # Asset directory browser
    AssetDetailPage/        # Instrument detail with charts, stats,
news
    AnalysisPage/           # Research terminal
    ForksPage/              # Pitchfork analysis browser
    AlertsPage/             # Alert management
    SettingsPage/           # App configuration
  components/
    ChartWorkspace.jsx      # Bloomberg-style terminal chart
    AIChatSidebar.jsx       # Ollama chat interface
    SchedulerPanel.jsx      # Background job management
    PitchforkLabPanel.jsx   # Pitchfork analysis controls
    TopNavigation.jsx       # Header navigation
    TerminalPivotRail.jsx   # Quick-access ticker rail
    ResearchMlLab.jsx       # ML signals panel
    FundamentalRibbon.jsx   # Key metrics display
    ForkChartThumb.jsx      # Pitchfork preview thumbnail
  watchlist/
    WatchlistsDashboard.jsx # Watchlist management
    WatchlistInsights.jsx   # Watchlist analytics
    ResearchMacroLab.jsx    # Macro research tools
    watchlist.css            # Shared watchlist/card styles
  utils/
    constants.js            # API base URL, localStorage keys,
defaults
    math.js                 # EMA, SMA, Pearson, Z-score, pitchfork
math
    portfolio.js            # Portfolio normalization, ledger
helpers
  styles/
    design-system.css       # Token foundation (spacing,
typography, motion)
    shared.css              # Theme tokens, component primitives,
chart styles
    layout.css              # Page layout, sidebar, content area
```

Module Routing (App.js)

The app uses a simple `useState('dashboard')` for module switching · no React Router. Modules:

- `dashboard` · HomeDashboard
- `portfolio` · PortfolioDashboard
- `watchlists` · WatchlistsDashboard
- `assets` · UniversePage
- `asset-detail` · AssetDetailPage
- `analysis` · AnalysisPage (with ChartWorkspace)
- `forks` · ForksPage
- `alerts` · AlertsPage

- `settings` · SettingsPage

Query params: `?symbol=AAPL`, `?module=forks`, `?fork=1`

useQuantEngine Hook

The central state management hook manages:

- **View state**: viewMode, theme, loading flags
- **Market data**: tickersData, ohlcData, optionsData, tickerDetails
- **Terminal state**: selectedTicker, currentTimeframe, chartZoom, selectedCandle
- **Technical analysis**: showVolume, showEMA20, showSMA50, showSMA200, showPitchfork, detectedPivots
- **Screener**: isScreening, screenerResults, screenerProgress
- **Portfolio**: portfolios, transactions, holdings, snapshots
- **Watchlist**: custom lists, automation config
- **AI**: chat history, suggestions, LLM config

Key functions: `openTerminal()`, `findForkInAll()`, `resetZoom()`, `setSelectedCandle()`

ChartWorkspace Component

Bloomberg-inspired terminal chart with:

- **Header bar**: Symbol, current price, period return, timeframe chips
- **Toolbar**: Chart style selector (Candle/Mountain/Line/Hybrid), Events toggle, Indicators panel, Fork toggle, Scan All button
- **OHLC readout strip**: Live O/H/L/C/Vol display on hover
- **Chart canvas**: ApexCharts with theme-aware palettes, crosshairs, custom tooltip
- **Scanner results**: Compact fork-hit display

Theme palettes:

- Dark: Teal price, purple EMA, gold SMA50, blue SMA200, green/red candles
- Light: Forest green price, violet EMA, amber SMA50, sapphire SMA200

---

6. Data Pipeline

fetch_all_tickers.py · Universe Builder

Builds `all_global_tickers.json` by aggregating:

1. **NSE Equity**: CSV from nsearchives.nseindia.com
2. **NSE FO**: Futures and options CSV
3. **BSE Equity**: All equity groups via `api.bseindia.com` (4838 symbols) with browser-like headers
4. **NSE Index Options**: Proxy symbols for Nifty/BankNifty
5. **US Markets**: S&P 500 and NASDAQ 100 from Wikipedia tables
6. **Global Indices**: FTSE 100 via PyTickerSymbols
7. **Forex / Commodities**: Hardcoded Yahoo Finance symbols
8. **Indian Mutual Funds**: AMFI scheme registry

Output: JSON mapping category names to symbol arrays.

market_download.py · Data Fetcher

- `parquet_symbol_key(symbol)`: Converts symbols to filesystem-safe keys (e.g., `MF:123` -> `MF_123`)
- `download_ticker_data(symbol, intervals)`: Downloads historical data for specified intervals
- `download_eod_bar(symbol)`: Appends latest daily bar to Parquet + SQLite
- `download_intraday_session(symbol)`: Appends 15-minute bars
- `batch_fetch_live_quotes(symbols)`: Bulk current prices via yfinance
- `download_symbols(symbols, intervals)`: Batch download with progress

Storage layout:

```
local_market_data/
  ohlc.sqlite          # Centralized OHLC database
  15m/                 # 15-minute Parquet files
  1h/                  # 1-hour Parquet files
  1d/                  # Daily Parquet files
  1wk/                 # Weekly Parquet files
```

market_universe.py · Universe Resolver

- `load_tickers()`: Loads and validates `all_global_tickers.json`
- `symbol_profile(symbol)`: Returns asset family, region, exchange, categories
- `search_local_instruments(query)`: Fuzzy search across universe
- `is_market_open(exchange)`: Exchange calendar integration
- `EXCHANGE_SCHEDULE`: Trading hours for NSE, NYSE, LSE, etc.
- `INDEX_PROXY_MAP`: Maps index symbols to their Yahoo Finance proxy

---

7. Context Modules

context/wikipedia_client.py · Wikipedia Enrichment

- `wikipedia_enrichment_from_yahoo(info, symbol)`: Uses Yahoo info hints (longName, shortName) to search Wikipedia via MediaWiki API
- Returns: `wikipedia_title`, `wikipedia_extract`, `wikipedia_url`
- Used by both `_fetch_and_cache_instrument_info()` and `_wiki_for_ticker()`

context/india_mutual_funds.py · MF Data

- `refresh_scheme_registry()`: Downloads AMFI scheme list
- `refresh_nav_history(scheme_code)`: Fetches NAV history from mfapi.in
- `get_mutual_fund_details(symbol)`: Returns fund metadata + latest NAV
- MF symbols use `MF:` prefix convention

context/india_gold_rates.py · Precious Metals

- Scrapes goodreturns.in for gold/silver rates by Indian city
- Returns structured pricing data for display

context/consumer_*.py · Consumer Correlation

- `consumer_rag.py`: Builds SQLite chunks table with Ollama embeddings
- `consumer_query.py`: RAG query over consumer cases with LLM
- `consumer_correlation.py`: Pearson/Spearman correlation between consumer case time series and OHLC data
- `consumer_resolve.py`: Maps company names to Yahoo ticker symbols

context/news_store.py · News Aggregation

- Fetches per-symbol news from yfinance
- Optional: Finnhub and NewsAPI integration via environment keys

context/open_context.py · Open Feeds

- RSS feeds via feedparser
- Reddit public JSON listings
- CourtListener legal search API
- Outputs: `ledger.jsonl`

context/portfolio_*.py · Portfolio Analytics

- `portfolio_ledger.py`: Transaction normalization, holdings derivation
- `portfolio_reports.py`: Analytics, tax summaries, fee summaries, copilot context
- `portfolio_fee_registry.py`: Indian brokerage fee estimation
- `portfolio_import.py`: CSV import with preview/commit workflow

context/watchlist_store.py · Persistence

- Reads/writes `user_settings.json` for watchlist and portfolio data
- Simple file-based persistence (no external database)

---

8. Agent System (Ollama)

agents/download_agent.py

Ollama-based chat agent with tool-calling capability for automated Parquet downloads. Accepts natural language instructions like "Download daily data for AAPL and MSFT."

agents/context_agent.py

Ollama agent for contextual research: can fetch news, open context, and OHLC data through tool-calling.

agents/download_tools.py & context_tools.py

Tool specifications and dispatch functions that bridge Ollama function calls to actual data pipeline operations.

Configuration

- Ollama endpoint: `http://127.0.0.1:11434`

- Chat model: configurable via `OLLAMA_MODEL`
- Embedding model: configurable via `OLLAMA_EMBED_MODEL`

---

9. External APIs and Services

Service	Module	Usage
Yahoo Finance	yfinance, market_download	OHLC, quotes, info, options, news
Yahoo Search API	backend/app.py	Instrument search (`query2.finance.yahoo.com/v1/finance/search`)
Wikipedia MediaWiki	context/wikipedia_client.py	Title/extract enrichment
Wikipedia Tables	fetch_all_tickers.py, fetch_sp500.py	S&P 500, NASDAQ 100 constituent lists
NSE India	fetch_all_tickers.py	Equity and F&O CSVs from nsearchives.nseindia.com
BSE India	fetch_all_tickers.py	Active equity listings via api.bseindia.com
AMFI India	context/india_mutual_funds.py	Mutual fund scheme registry
MFapi.in	context/india_mutual_funds.py	Mutual fund NAV history
GoodReturns.in	context/india_gold_rates.py	Gold/silver city rates (scraped)
Upstox	backend/app.py	Option chain data for NSE indices (when token configured)
Ollama	backend/app.py, agents/	Local LLM for analysis, chat, embeddings
Finnhub	context/news_store.py	Financial news (optional, requires API key)
NewsAPI	context/news_store.py	General news (optional, requires API key)
Reddit	context/open_context.py	Public JSON listings from relevant subreddits
CourtListener	context/open_context.py	Legal case search (optional, requires API key)
PyTickerSymbols	fetch_all_tickers.py	FTSE 100, Nikkei, Hang Seng, DAX constituents

---

10. Database Schemas

ohlc.sqlite · Market Data Store

```
CREATE TABLE IF NOT EXISTS ohlc (
    symbol   TEXT NOT NULL,
    interval TEXT NOT NULL,
    ts       TEXT NOT NULL,
    open     REAL,
    high     REAL,
    low      REAL,
    close    REAL,
    volume   REAL,
    PRIMARY KEY (symbol, interval, ts)
);
CREATE INDEX IF NOT EXISTS idx_ohlc_sym_int_ts ON ohlc(symbol,
interval, ts);
```

instrument_info · Instrument Metadata Cache

```
CREATE TABLE IF NOT EXISTS instrument_info (
    symbol     TEXT PRIMARY KEY,
    info_json  TEXT,
    fetched_at TEXT
);
CREATE INDEX IF NOT EXISTS idx_info_fetched ON
instrument_info(fetched_at);
```

The `info_json` column stores merged Yahoo Finance info + Wikipedia enrichment as JSON.

rag.sqlite · Consumer RAG Store

```
CREATE TABLE chunks (
    chunk_id     TEXT PRIMARY KEY,
    case_id      TEXT,
    chunk_idx    INTEGER,
    text         TEXT,
    symbol       TEXT,
    published_at TEXT,
    dim          INTEGER,
    embedding    BLOB
);
```

scheduler_jobs.sqlite

Managed by APScheduler's SQLAlchemy job store. Contains serialized job definitions.

Parquet Files

Each file at `local_market_data/{interval}/{safe_symbol_key}.parquet` contains columns: `Date` (index), `Open`, `High`, `Low`, `Close`, `Volume`.

---

11. Design System

Token Foundation (design-system.css)

Spacing: 4px base ramp (--md-space-1 through --md-space-10)
Typography: Plus Jakarta Sans (UI), JetBrains Mono (data/code)
Radius: xs(4px), sm(8px), md(12px), lg(16px), xl(24px)
Motion: Material M3-inspired curves (std 200ms, emphasis 400ms)
Elevation: 5-level shadow scale
Z-index: base(0), sticky(10), dropdown(20), overlay(30), modal(40), toast(50)

Theme System (shared.css)

Four themes, each defining ~35 CSS custom properties:

- **Light**: Clean white surfaces, indigo accent (#6366f1)
- **Dark (Midnight)**: Deep charcoal (#13141b), purple accent (#818cf8)
- **Ocean**: Navy blue (#111c32), cyan accent (#38bdf8)
- **Sand**: Warm parchment (#f5f2ec), amber accent (#b45309)

Component Primitives

Cards: Flat-bordered with 1px solid borders, no default shadows, `border-radius-md`, tight padding (`space-3`). Hover adds `elevation-1`.

Chips/Pills: Monospace, uppercase, `border-radius-xs`, transparent background, 0.625rem font size.

Tabs: Terminal-style with monospace font, 3px gaps, minimal radius.

Tables: Monospace headers (0.56rem), sticky header row, alternating row hover.

Chart Styles (Bloomberg Terminal)

- Shell with flex column layout (header -> toolbar -> readout -> canvas)
- Header: symbol + readout + timeframe chips
- Toolbar: chart style, indicators, scanner controls
- OHLC readout: compact strip with O/H/L/C/Vol on hover
- Theme-adaptive palettes (teal/forest for dark/light)
- Custom tooltip with day-over-day change display

---

12. API Reference

Ticker & Market Data

| Method | Endpoint | Description | |--------|----------|----------| | GET | `/api/tickers` | Full universe categories with symbols | | GET | `/api/tickers/summary` | Category-level count summary | | GET | `/api/tickers/presets` | Curated preset collections | | GET | `/api/search/instruments` | Full-text instrument search | | GET | `/api/ticker/{symbol}` | Comprehensive instrument details (incl. Wikipedia) | | GET | `/api/ticker/{symbol}/ohlc` | OHLC time series (query: `timeframe`) | | GET | `/api/ticker/{symbol}/options` | Options chain data | | POST | `/api/ticker/{symbol}/download` | On-demand Parquet download | | POST | `/api/ticker/{symbol}/download-full-db` | Full SQLite archive download | | GET | `/api/instrument/{symbol}/info` | Cached instrument info (Yahoo + Wikipedia) |

Mutual Funds & Metals

| Method | Endpoint | Description | |--------|----------|----------| | GET | `/api/mf/categories` | MF category taxonomy | | GET | `/api/mf/browse` | Browse MF schemes with filters | | GET | `/api/gold-rates` | Gold rates (India) | | GET | `/api/silver-rates` | Silver rates (India) | | GET | `/api/precious-metals/india` | Combined precious metals data | | GET | `/api/macro/snapshot` | Macro overview data |

Portfolio

| Method | Endpoint | Description | |--------|----------|----------| | GET | `/api/portfolios` | All portfolios | | PUT | `/api/portfolios` | Save portfolios | | GET | `/api/portfolio/fee-registry` | Indian fee schedules | | POST | `/api/portfolio/fee-preview` | Estimate transaction charges | | GET | `/api/portfolio/analytics` | Portfolio analytics | | POST | `/api/portfolio/import/preview` | CSV import preview | | POST | `/api/portfolio/import/commit` | Commit CSV import | | GET | `/api/portfolio/report/tax-summary` | Tax report | | GET | `/api/portfolio/report/fee-summary` | Fee report | | POST | `/api/portfolio/copilot/context` | AI copilot context bundle |

Watchlist

| Method | Endpoint | Description | |--------|----------|----------| | GET | `/api/watchlist` | Get watchlist | | PUT | `/api/watchlist` | Update watchlist | | GET | `/api/watchlist/summary` | Watchlist summary with quotes | | POST | `/api/instruments/batch-quote` | Batch live quotes |

Scheduler

| Method | Endpoint | Description | |--------|----------|----------| | GET | `/api/scheduler/status` | Scheduler state + job list | | POST | `/api/scheduler/start` | Start scheduler | | POST | `/api/scheduler/stop` | Stop scheduler | | POST | `/api/scheduler/job/{job_id}/trigger` | Manual job trigger | | POST |

`/api/scheduler/job/{job_id}/pause` | Pause job | | POST | `/api/scheduler/job/{job_id}/resume` | Resume job | | GET | `/api/scheduler/logs` | Recent scheduler logs | | GET | `/api/scheduler/live-quotes` | Cached live quotes |

AI & Agents

| Method | Endpoint | Description | |--------|----------|-------------| | POST | `/api/ai/analyze` | Ollama LLM analysis | | POST | `/api/agents/download` | Download agent (tool-calling) | | POST | `/api/agents/context-run` | Context research agent |

Context

| Method | Endpoint | Description | |--------|----------|-------------| | POST | `/api/context/consumer/ingest` | Ingest consumer cases | | POST | `/api/context/consumer/reindex` | Rebuild RAG embeddings | | POST | `/api/context/consumer/query` | RAG query | | GET | `/api/context/consumer/preview/{symbol}` | Consumer preview | | POST | `/api/context/news/refresh` | Refresh news cache | | GET | `/api/context/news/{symbol}` | Get cached news | | POST | `/api/context/open/ingest` | Ingest open feeds | | GET | `/api/context/unified/{symbol}` | Unified context bundle |

Admin

| Method | Endpoint | Description | |--------|----------|-------------| | POST | `/api/admin/nuke-local-data` | Delete local data | | POST | `/api/admin/bulk-info-load` | Batch instrument info loading | | GET | `/api/admin/bulk-info-load-status/{job_id}` | Bulk load status | | POST | `/api/admin/redownload-all` | Re-download all market data | | GET | `/api/admin/redownload-status/{job_id}` | Redownload status | | POST | `/api/admin/reset-and-redownload` | Nuke + redownload | | POST | `/api/admin/download-all-and-calculate` | Full pipeline run | | GET | `/api/admin/download-all-and-calculate-status/{job_id}` | Pipeline status |

WebSocket

| Endpoint | Description | |----------|-------------| | `/ws/live/{symbol}` | Simulated live tick stream |

---

13. Configuration and Environment

Start Scripts

- `start-dashboard.sh`: Launches React dev server (`npm start`)
- `run_backend.sh`: Activates venv, starts uvicorn on `127.0.0.1:8000`
- `dev_with_ollama.sh`: Starts backend with Ollama service
- `start_terminal.sh`: Terminal-mode launcher

Environment Variables

| Variable | Default | Description | |----------|---------|-------------| | `TICKERS_JSON` | `all_global_tickers.json` | Universe file path | | `MARKET_DATA_DIR` | `local_market_data` | Data storage directory | | `REACT_APP_API_BASE` | `http://localhost:8000` | API URL for frontend | | `OLLAMA_HOST` | `http://127.0.0.1:11434` | Ollama endpoint | | `OLLAMA_MODEL` | (varies) | LLM model name | | `OLLAMA_EMBED_MODEL` | (varies) | Embedding model name | | `FINNHUB_API_KEY` | (none) | Finnhub news API key | | `NEWSAPI_KEY` | (none) | NewsAPI key | | `COURTLISTENER_API_KEY` | (none) | Legal search

API key | | `USER_SETTINGS_PATH` | `user_settings.json` | Watchlist/portfolio persistence |

Python Dependencies (requirements.txt)

```
eval-type-backport>=0.2.0
fastapi>=0.115.0
uvicorn[standard]>=0.32.0
pydantic>=2.0.0
requests>=2.28.0
yfinance>=0.2.0
pandas>=2.0.0
lxml
numpy>=1.24.0
pyarrow>=14.0.0
tqdm>=4.65.0
feedparser>=6.0.0
pytickersymbols
beautifulsoup4>=4.12.0
APScheduler>=3.10.0
SQLAlchemy>=2.0.0
exchange_calendars>=4.5.0
holidays>=0.50
```

NPM Dependencies (package.json)

```
react: ^19.2.4
react-dom: ^19.2.4
react-scripts: 5.0.1
apexcharts: ^5.10.3
react-apexcharts: ^2.1.0
recharts: ^3.8.0
```

---

14. Changelog · Recent Session Work

Session: March 2026 · Bloomberg Redesign + Ticker Hardening + Wikipedia

Chart Redesign (Bloomberg Terminal Style)

- **ChartWorkspace.jsx**: Full rewrite with structured terminal layout
  - Header bar: symbol, price readout, period return, timeframe chips
  - Toolbar row: chart style selector, indicator toggles, scanner controls
  - OHLC readout strip: updates on hover with O/H/L/C/Vol/date
  - Theme-aware palettes (PALETTE_DARK / PALETTE_LIGHT)
  - Custom tooltip with day-over-day change and percentage
  - ApexCharts toolbar hidden in favor of custom terminal chrome
  - Chart defaults to candle view instead of mountain

Card Primitives & Page Density

- **shared.css**: `.md-card` shifted to flat borders, no default shadows, tighter padding

- **watchlist.css**: `.mdl-card`, `.mdl-hero`, `.mdl-pill` tightened
- **UniversePage.css**: Asset tiles denser (3px gap, accent-border hover), terminal-style type chips
- **AssetDetailPage.css**: Monospace throughout, compact tabs/chips, structured alert boxes
- **AnalysisPage.css**: Terminal-style tab chips

Ticker Hardening (Backend)

- `_safe_round()` for all numeric fields · handles NaN, None, non-numeric
- Returns `null` instead of `"N/A"` for missing values
- New fields: `eps`, `dividendYield`, `beta`, `volume`, `avgVolume`
- `previousClose` field name corrected (legacy `prevClose` alias kept)
- News fetching wrapped in try/catch
- `logger.warning` on failure instead of silent exception

Wikipedia Integration

- `_wiki_for_ticker()`: Checks cached instrument_info, falls back to live API
- Returns `wikipediaTitle`, `wikipediaExtract`, `wikiUrl`
- Frontend About card shows Wikipedia title as section header
- Collapsible "Yahoo Finance summary" when both sources available
- Description field uses Wikipedia extract as primary, Yahoo as fallback

BSE India Fix (Earlier in Session)

- BSE API endpoint updated to `https://api.bseindia.com/BseIndiaAPI/api/ListofScripData/w`
- Required headers added: Origin, Referer, Accept: application/json
- Iterates all BSE equity groups: A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, T, X, XC, XD, XT, Z
- Symbol parsing handles `&` and `-` in ticker names (e.g., `M&M.BO`)
- Result: 4838 BSE symbols (up from 30 fallback)

Chart Loading & Download Fixes (Earlier in Session)

- `.ad-chart-wrap` given explicit height for ApexCharts rendering
- `useQuantEngine.js`: `openTerminal` now validates `Array.isArray(rawOhlc)` and `ohlcRes.ok`
- `ChartWorkspace.jsx`: `skipViewMode` prop prevents view mode changes when embedded
- Download feedback improved with structured status/records_saved/error messages
- Parquet paths use `parquet_symbol_key()` consistently across all read/write paths

---

End of Document